

学习资源

Coding Agent 工作原理

 Copy page

了解 Agent 循环机制、内置工具，以及 Coding Agent 如何与项目进行交互。

Coding Agent 通常以终端 CLI 或 IDE 插件的形式运行，例如 Claude Code、Codex、Cline、Cursor、Copilot 等工具。针对 Coding Agent 的高频调用需求，[GLM Coding Plan](#) 提供了更适合编程场景的模型额度与调用成本。

虽然 Coding Agent 主要用于编程相关任务，但实际上，只要是能够通过命令行或开发环境完成的工作，通常都可以提供协助，例如：

- 编写与维护技术文档
- 执行构建与测试流程
- 搜索和分析项目代码
- 调研技术问题或查询文档
- 运行脚本或系统命令

本文将介绍 Coding Agent 的设计思路。在此基础上，一些工具还提供了扩展组件，如 **技能 (Skills)**、**外部服务连接 (MCP)**，用于进一步增强 Agent 的能力边界。

Agent 循环 (Agentic Loop)

当你向 Coding Agent 提出一个任务时，它通常会在以下三个阶段之间循环工作：

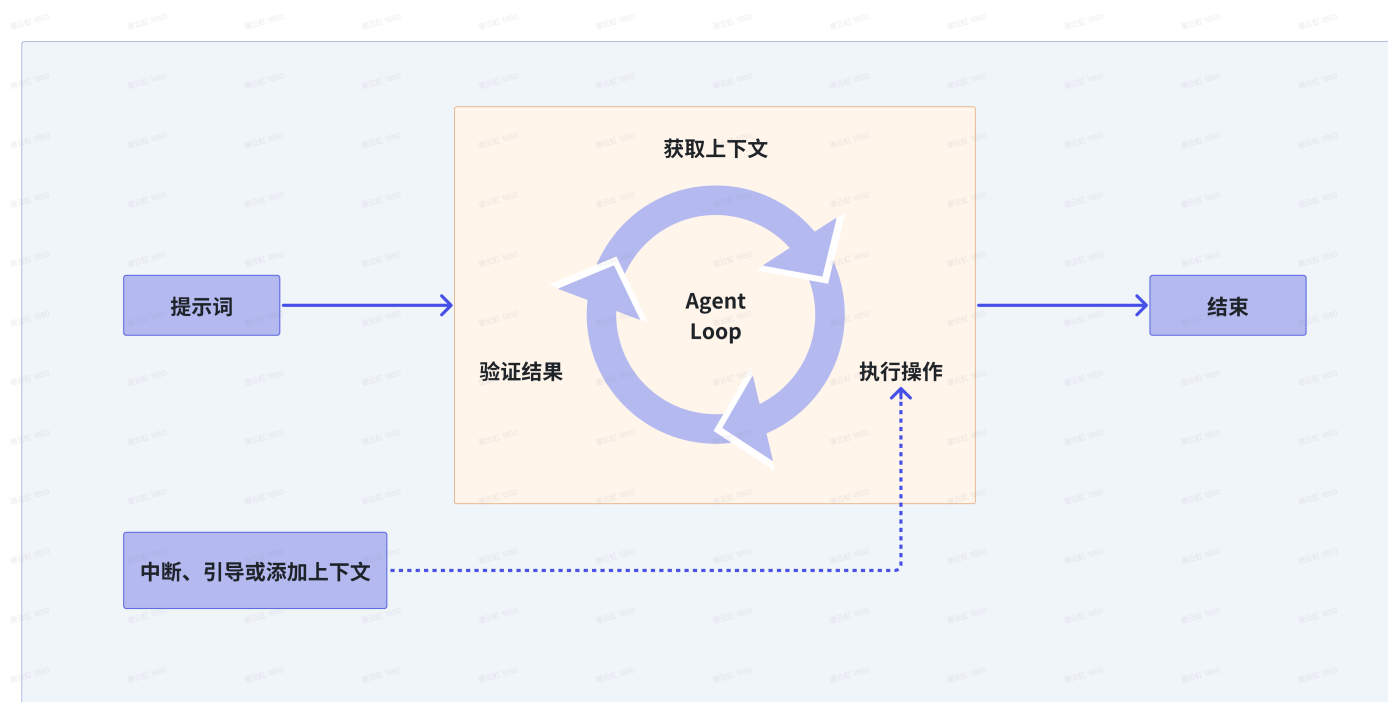
💡 获取上下文 → 执行操作 → 验证结果

这三个阶段并不是严格按顺序执行，而是在整个任务过程中不断交替出现。

在这个过程中，Coding Agent 会持续调用各种工具，例如：



- 搜索代码以理解项目结构
- 修改文件实现需求
- 运行测试验证修改结果



这个循环会根据任务类型进行动态调整。例如：

Case 1: 代码结构理解 **Case 2: Bug 修复** **Case 3: 大型重构**

如果只是询问代码结构，通常只需要完成上下文获取与代码阅读，不一定涉及修改和执行。

Coding Agent 会根据每一步获得的信息不断调整策略，在执行过程中串联多个操作，并持续修正方向。用户同样是这个循环的一部分。在任何时候，你都可以：



- 中断当前任务
- 提供新的上下文信息
- 指示它尝试新的解决思路

Coding Agent 可以自主执行任务，但始终会响应你的输入。



- **模型 (Models)** ：负责理解任务与推理决策
- **工具 (Tools)** ：负责执行实际操作

大多数 Coding Agent 都会在语言模型之上提供一层 Agent 运行框架，用于管理上下文、调度工具并执行任务，从而将语言模型转化为一个能够实际操作代码库的编程 Agent。

大模型 (Model)

在 Coding Agent 架构中，大模型负责：

- 阅读代码并理解项目结构
- 推理代码逻辑
- 规划任务执行步骤
- 根据执行结果进行调整



模型通常能够：

- 解析多种编程语言
- 理解模块之间的依赖关系
- 推断实现目标所需的修改

在复杂任务中，模型会将任务拆分为多个步骤，并逐步执行。

工具 (Tool)

如果没有工具，大模型只能生成文本。Coding Agent 的关键在于 **工具系统 (Tool)** ，使模型能够执行实际操作。常见工具类型包括：

类别	能力
文件操作	读取文件、修改代码、创建新文件、重命名或调整结构

类别	能力
BigModel	
搜索	按模式查找文件、使用正则搜索内容、浏览代码库
命令执行	运行 Shell 命令、启动服务、运行测试、使用 Git
Web	搜索网页、获取文档、查询错误信息
代码分析	查看类型错误、跳转定义、查找引用（需代码智能插件）

每一次工具调用都会产生新的信息，并反馈给模型，用于指导下一步决策。这正是 **Agentic Loop** 的运行过程。

扩展组件

内置工具只是基础能力。你可以通过以下方式扩展 Coding Agent 的能力：

- **Skills**：封装常见工作流程
- **MCP**：连接外部服务
- **Hooks**：自动化任务流程
- **Subagents**：将任务委托给子 Agent 执行

这些扩展能力构成了 **Agent 循环之上的能力层**。关于各类扩展组件的具体能力与使用方式，可参考 [Agentic 扩展组件](#)。

Coding Agent 可以访问的环境

当 Coding Agent 在某个项目中运行时，它通常可以访问以下信息：

▼ 项目代码

- 当前目录中的所有代码文件
- 项目结构和配置

- 命令行工具
- 构建工具
- 包管理器
- Git

▼ 版本控制信息

- 当前分支
- 未提交修改
- 最近提交记录

由于 Coding Agent 可以访问整个项目，因此能够在多个文件之间进行协同修改，而不仅仅局限于当前文件。

项目级配置文件

许多 Coding Agent 工具允许通过 **项目级配置文件** 提供额外上下文，例如：

- 项目规范
- 编码规则
- 常用命令
- 项目结构说明

不同厂商通常采用不同文件名称。

工具	配置文件
Claude Code	CLAUDE.md
Cursor	.cursor/rules

工具	BigModel	配置文件	🔍 ⋮
Cline		.cline/rules	
Codex	>	AGENTS.md	



它们的作用是给 Coding Agent 提供 长期项目上下文和开发规则。不同工具通常只会读取自身生态中的配置文件。

运行环境

Coding Agent 可以在不同环境中运行：

环境	代码执行位置
本地	用户机器
云端	云端虚拟机
远程控制	本地机器，由远程界面控制

不同工具的实现方式可能不同，但底层运行机制基本一致。

会话与上下文管理

Coding Agent 在执行任务时需要维护上下文，包括：

- 对话历史
- 文件内容
- 命令输出
- 项目规则

随着任务进行，上下文会不断增加。多数系统会自动进行 上下文压缩（Context Compression），例如：

💡 对于长期规则，通常建议写入 **项目级配置文件**，而不是依赖对话历史。

安全与权限控制

为了避免自动化操作带来风险，大多数 Coding Agent 都提供安全机制，例如：

[变更回滚](#) [权限控制](#)

在修改文件前创建快照，允许恢复到之前状态。

高效使用 Coding Agent

以下实践可以提升 Coding Agent 的使用效果：

1. **提供清晰任务**：越具体的任务描述，成功率越高。
2. **提供可验证目标**：例如测试用例或期望输出。
3. **复杂任务先规划**：先让 Agent 分析代码并制定执行计划。
4. **将任务委托给 Agent**：提供目标与上下文，而不是逐条指令。

学习资源



Agentic 扩展组件



记忆机制



常用工作流



最佳实践

